

```
In [2]: import sys  
print(sys.executable)
```

g:\ProyectosPython\proyecto3\.venv\Scripts\python.exe

# Proyecto 3 - Entorno y carga de datos

## Integrantes

- Daniel Fernando Salgado Santamaría
- Jairo Wladimir Jhayya Perlaza
- Luis Gabriel Salgado Santamaría
- Oscar Paul Naranjo Castro

**Fecha:** 2026-05-12

**Notebook:** 01-jj-entorno-y-carga.ipynb

## Objetivo

Configurar el entorno de trabajo del proyecto, definir rutas reproducibles, cargar los archivos fuente y realizar una validación inicial de estructura, dimensiones y tipos de datos.

## Archivos esperados

- customers.csv
- sessions.csv
- events.csv
- orders.csv
- order\_items.csv
- products.csv

- reviews.csv

```
In [18]: # Importa librerías base del proyecto para manejo de rutas, sistema, análisis y visualización.
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [19]: # Muestra la ruta exacta del intérprete de Python que está ejecutando el notebook.
print("Python ejecutándose desde:")
print(sys.executable)
```

Python ejecutándose desde:

g:\ProyectosPython\proyecto3\.venv\Scripts\python.exe

```
In [20]: # Configura opciones de visualización de pandas y el estilo general de gráficos con seaborn.
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)
pd.set_option("display.width", 120)

sns.set_theme(style="whitegrid", context="notebook")
```

```
In [21]: # Define las rutas principales del proyecto y crea carpetas necesarias si no existen.

PROJECT_ROOT = Path.cwd().resolve().parent if Path.cwd().name == "notebooks" else Path.cwd().resolve()

DATA_DIR = PROJECT_ROOT / "data"
RAW_DIR = DATA_DIR / "raw"
INTERIM_DIR = DATA_DIR / "interim"
PROCESSED_DIR = DATA_DIR / "processed"

REPORTS_DIR = PROJECT_ROOT / "reports"
FIGURES_DIR = REPORTS_DIR / "figures"
TABLES_DIR = REPORTS_DIR / "tables"

for path in [RAW_DIR, INTERIM_DIR, PROCESSED_DIR, FIGURES_DIR, TABLES_DIR]:
    path.mkdir(parents=True, exist_ok=True)
```

```
print("PROJECT_ROOT:", PROJECT_ROOT)
print("RAW_DIR:", RAW_DIR)
print("FIGURES_DIR:", FIGURES_DIR)
print("TABLES_DIR:", TABLES_DIR)
```

PROJECT\_ROOT: G:\ProyectosPython\proyecto3\desarrollo3

RAW\_DIR: G:\ProyectosPython\proyecto3\desarrollo3\data\raw

FIGURES\_DIR: G:\ProyectosPython\proyecto3\desarrollo3\reports\figures

TABLES\_DIR: G:\ProyectosPython\proyecto3\desarrollo3\reports\tables

## Verificación de archivos de entrada

En esta sección se comprueba si los archivos fuente existen en `data/raw/` antes de cargarlos.

In [38]: *# Verifica si Los archivos CSV esperados están presentes en La carpeta data/raw.*

```
expected_files = {
    "customers": "customers.csv",
    "sessions": "sessions.csv",
    "events": "events.csv",
    "orders": "orders.csv",
    "order_items": "order_items.csv",
    "products": "products.csv",
    "reviews": "reviews.csv",
}

file_status = pd.DataFrame({
    "dataset": expected_files.keys(),
    "filename": expected_files.values(),
})

file_status["exists"] = file_status["filename"].apply(lambda x: (RAW_DIR / x).exists())
file_status["full_path"] = file_status["filename"].apply(lambda x: str(RAW_DIR / x))

file_status
```

Out[38]:

|   | dataset     | filename        | exists | full_path                                         |
|---|-------------|-----------------|--------|---------------------------------------------------|
| 0 | customers   | customers.csv   | True   | G:\ProyectosPython\proyecto3\desarrollo3\data\... |
| 1 | sessions    | sessions.csv    | True   | G:\ProyectosPython\proyecto3\desarrollo3\data\... |
| 2 | events      | events.csv      | True   | G:\ProyectosPython\proyecto3\desarrollo3\data\... |
| 3 | orders      | orders.csv      | True   | G:\ProyectosPython\proyecto3\desarrollo3\data\... |
| 4 | order_items | order_items.csv | True   | G:\ProyectosPython\proyecto3\desarrollo3\data\... |
| 5 | products    | products.csv    | True   | G:\ProyectosPython\proyecto3\desarrollo3\data\... |
| 6 | reviews     | reviews.csv     | True   | G:\ProyectosPython\proyecto3\desarrollo3\data\... |

In [39]: *# Carga todos Los archivos CSV desde data/raw a DataFrames de pandas.*

```

customers = pd.read_csv(RAW_DIR / "customers.csv")
sessions = pd.read_csv(RAW_DIR / "sessions.csv")
events = pd.read_csv(RAW_DIR / "events.csv")
orders = pd.read_csv(RAW_DIR / "orders.csv")
order_items = pd.read_csv(RAW_DIR / "order_items.csv")
products = pd.read_csv(RAW_DIR / "products.csv")
reviews = pd.read_csv(RAW_DIR / "reviews.csv")

```

## Resumen inicial de dimensiones

In [24]: *# Agrupa todos los datasets en un diccionario y genera un resumen de tamaño y uso de memoria.*

```

datasets = {
    "customers": customers,
    "sessions": sessions,
    "events": events,
    "orders": orders,
    "order_items": order_items,
    "products": products,
    "reviews": reviews,
}

```

```
summary = pd.DataFrame([
    {
        "dataset": name,
        "rows": df.shape[0],
        "columns": df.shape[1],
        "memory_mb": round(df.memory_usage(deep=True).sum() / (1024**2), 2)
    }
    for name, df in datasets.items()
]).sort_values("rows", ascending=False)

summary
```

Out[24]:

|   | dataset     | rows   | columns | memory_mb |
|---|-------------|--------|---------|-----------|
| 2 | events      | 760958 | 10      | 156.19    |
| 1 | sessions    | 120000 | 6       | 28.10     |
| 4 | order_items | 59163  | 5       | 2.26      |
| 3 | orders      | 33580  | 10      | 10.34     |
| 0 | customers   | 20000  | 7       | 4.96      |
| 6 | reviews     | 10780  | 6       | 1.80      |
| 5 | products    | 1197   | 6       | 0.18      |

## Muestra inicial de los datasets

Se visualiza una muestra de cada tabla para verificar estructura, nombres de columnas y contenido general.

```
In [25]: # Muestra las primeras filas del dataset de clientes.
customers.head()
```

Out[25]:

|   | customer_id | name             | email                      | country | age | signup_date | marketing_opt_in |
|---|-------------|------------------|----------------------------|---------|-----|-------------|------------------|
| 0 | 1           | Jennifer Salinas | nicholas59@example.org     | JP      | 71  | 2020-09-04  | True             |
| 1 | 2           | Phillip Ramos    | christinarubio@example.com | IN      | 26  | 2020-04-05  | False            |
| 2 | 3           | Dawn Fowler      | jessica03@example.org      | BR      | 21  | 2023-08-31  | True             |
| 3 | 4           | Mario Butler     | paula27@example.org        | FR      | 63  | 2022-06-30  | True             |
| 4 | 5           | Amber Brown      | kevin85@example.net        | BR      | 19  | 2022-07-22  | True             |

In [26]:

```
# Muestra las primeras filas del dataset de sesiones.  
sessions.head()
```

Out[26]:

|   | session_id | customer_id | start_time          | device  | source  | country |
|---|------------|-------------|---------------------|---------|---------|---------|
| 0 | 1          | 12360       | 2021-12-27T00:01:36 | mobile  | email   | DE      |
| 1 | 2          | 13917       | 2025-01-31T21:29:42 | desktop | organic | PL      |
| 2 | 3          | 1022        | 2024-02-19T00:52:50 | tablet  | organic | FR      |
| 3 | 4          | 2882        | 2024-08-04T19:54:31 | mobile  | direct  | GB      |
| 4 | 5          | 1286        | 2022-06-28T13:58:08 | desktop | email   | ES      |

In [27]:

```
# Muestra las primeras filas del dataset de eventos.  
events.head()
```

Out[27]:

|   | event_id | session_id | timestamp           | event_type  | product_id | qty | cart_size | payment | discount_pct | amount_usd |
|---|----------|------------|---------------------|-------------|------------|-----|-----------|---------|--------------|------------|
| 0 | 1        | 1          | 2021-12-27T00:08:36 | page_view   | 93.0       | NaN | NaN       | NaN     | NaN          | NaN        |
| 1 | 2        | 1          | 2021-12-27T00:16:36 | page_view   | 1005.0     | NaN | NaN       | NaN     | NaN          | NaN        |
| 2 | 3        | 1          | 2021-12-27T00:18:01 | add_to_cart | 1005.0     | 1.0 | NaN       | NaN     | NaN          | NaN        |
| 3 | 4        | 1          | 2021-12-27T00:45:36 | page_view   | 918.0      | NaN | NaN       | NaN     | NaN          | NaN        |
| 4 | 5        | 1          | 2021-12-27T01:03:36 | page_view   | 946.0      | NaN | NaN       | NaN     | NaN          | NaN        |

In [28]:

```
# Muestra las primeras filas del dataset de órdenes.  
orders.head()
```

Out[28]:

|   | order_id | customer_id | order_time          | payment_method | discount_pct | subtotal_usd | total_usd | country | device  | source  |
|---|----------|-------------|---------------------|----------------|--------------|--------------|-----------|---------|---------|---------|
| 0 | 1        | 13917       | 2025-01-31T23:07:42 | card           | 20           | 107.15       | 85.72     | PL      | desktop | organic |
| 1 | 2        | 1022        | 2024-02-19T01:17:50 | card           | 0            | 116.17       | 116.17    | FR      | tablet  | organic |
| 2 | 3        | 6145        | 2024-12-04T20:24:13 | card           | 0            | 137.35       | 137.35    | US      | mobile  | organic |
| 3 | 4        | 3152        | 2024-07-17T08:50:47 | card           | 15           | 32.18        | 27.35     | BR      | mobile  | email   |
| 4 | 5        | 12378       | 2020-08-21T16:54:16 | card           | 0            | 238.09       | 238.09    | NL      | desktop | paid    |

In [29]:

```
# Muestra las primeras filas del dataset de ítems por orden.  
order_items.head()
```

Out[29]:

|   | order_id | product_id | unit_price_usd | quantity | line_total_usd |
|---|----------|------------|----------------|----------|----------------|
| 0 | 1        | 226        | 107.15         | 1        | 107.15         |
| 1 | 2        | 771        | 116.17         | 1        | 116.17         |
| 2 | 3        | 415        | 94.49          | 1        | 94.49          |
| 3 | 3        | 24         | 42.86          | 1        | 42.86          |
| 4 | 4        | 1157       | 32.18          | 1        | 32.18          |

```
In [30]: # Muestra las primeras filas del dataset de productos.  
products.head()
```

Out[30]:

|   | product_id | category    | name                     | price_usd | cost_usd | margin_usd |
|---|------------|-------------|--------------------------|-----------|----------|------------|
| 0 | 1          | Electronics | SSD MediumBlue 149       | 570.28    | 352.69   | 217.59     |
| 1 | 2          | Electronics | Keyboard DeepPink 696    | 498.13    | 263.13   | 235.00     |
| 2 | 3          | Electronics | Headphones Orchid 188    | 548.53    | 309.60   | 238.93     |
| 3 | 4          | Electronics | Smartwatch BurlyWood 664 | 268.36    | 153.56   | 114.80     |
| 4 | 5          | Electronics | Smartwatch Cornsilk 328  | 63.69     | 42.65    | 21.04      |

```
In [31]: # Muestra las primeras filas del dataset de reseñas.  
reviews.head()
```



Out[31]:

|   | review_id | order_id | product_id | rating | review_text                          | review_time         |
|---|-----------|----------|------------|--------|--------------------------------------|---------------------|
| 0 | 1         | 4        | 1157       | 2      | Not great quality.                   | 2022-02-01T00:00:00 |
| 1 | 2         | 7        | 405        | 4      | Good value for money!                | 2020-03-07T00:00:00 |
| 2 | 3         | 7        | 487        | 5      | Excellent product, highly recommend! | 2024-12-26T00:00:00 |
| 3 | 4         | 7        | 442        | 5      | Excellent product, highly recommend! | 2021-03-18T00:00:00 |
| 4 | 5         | 7        | 348        | 4      | Good value for money!                | 2025-09-30T00:00:00 |

## Tipos de datos y valores nulos

```
In [35]: # Recorre cada dataset y muestra información general sobre columnas, tipos y valores no nulos.
for name, df in datasets.items():
    print("=" * 80)
    print(f"{name.upper()} - INFO")
    print("=" * 80)
    df.info()
    print("\n")
```

## =====

## CUSTOMERS - INFO

=====

&lt;class 'pandas.DataFrame'&gt;

RangeIndex: 20000 entries, 0 to 19999

Data columns (total 7 columns):

| # | Column           | Non-Null Count | Dtype |
|---|------------------|----------------|-------|
| 0 | customer_id      | 20000 non-null | int64 |
| 1 | name             | 20000 non-null | str   |
| 2 | email            | 20000 non-null | str   |
| 3 | country          | 20000 non-null | str   |
| 4 | age              | 20000 non-null | int64 |
| 5 | signup_date      | 20000 non-null | str   |
| 6 | marketing_opt_in | 20000 non-null | bool  |

dtypes: bool(1), int64(2), str(4)

memory usage: 957.2 KB

## =====

## SESSIONS - INFO

=====

&lt;class 'pandas.DataFrame'&gt;

RangeIndex: 120000 entries, 0 to 119999

Data columns (total 6 columns):

| # | Column      | Non-Null Count  | Dtype |
|---|-------------|-----------------|-------|
| 0 | session_id  | 120000 non-null | int64 |
| 1 | customer_id | 120000 non-null | int64 |
| 2 | start_time  | 120000 non-null | str   |
| 3 | device      | 120000 non-null | str   |
| 4 | source      | 120000 non-null | str   |
| 5 | country     | 120000 non-null | str   |

dtypes: int64(2), str(4)

memory usage: 5.5 MB

## =====

## EVENTS - INFO

=====

&lt;class 'pandas.DataFrame'&gt;

RangeIndex: 760958 entries, 0 to 760957

Data columns (total 10 columns):

| # | Column       | Non-Null Count  | Dtype   |
|---|--------------|-----------------|---------|
| 0 | event_id     | 760958 non-null | int64   |
| 1 | session_id   | 760958 non-null | int64   |
| 2 | timestamp    | 760958 non-null | str     |
| 3 | event_type   | 760958 non-null | str     |
| 4 | product_id   | 682469 non-null | float64 |
| 5 | qty          | 143126 non-null | float64 |
| 6 | cart_size    | 44909 non-null  | float64 |
| 7 | payment      | 33580 non-null  | str     |
| 8 | discount_pct | 33580 non-null  | float64 |
| 9 | amount_usd   | 33580 non-null  | float64 |

dtypes: float64(5), int64(2), str(3)

memory usage: 58.1 MB

=====

ORDERS - INFO

=====

<class 'pandas.DataFrame'>

RangeIndex: 33580 entries, 0 to 33579

Data columns (total 10 columns):

| # | Column         | Non-Null Count | Dtype   |
|---|----------------|----------------|---------|
| 0 | order_id       | 33580 non-null | int64   |
| 1 | customer_id    | 33580 non-null | int64   |
| 2 | order_time     | 33580 non-null | str     |
| 3 | payment_method | 33580 non-null | str     |
| 4 | discount_pct   | 33580 non-null | int64   |
| 5 | subtotal_usd   | 33580 non-null | float64 |
| 6 | total_usd      | 33580 non-null | float64 |
| 7 | country        | 33580 non-null | str     |
| 8 | device         | 33580 non-null | str     |
| 9 | source         | 33580 non-null | str     |

dtypes: float64(2), int64(3), str(5)

memory usage: 2.6 MB

=====

## ORDER\_ITEMS - INFO

```
=====
<class 'pandas.DataFrame'>
RangeIndex: 59163 entries, 0 to 59162
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   order_id        59163 non-null  int64
1   product_id      59163 non-null  int64
2   unit_price_usd  59163 non-null  float64
3   quantity        59163 non-null  int64
4   line_total_usd  59163 non-null  float64
dtypes: float64(2), int64(3)
memory usage: 2.3 MB
```

## PRODUCTS - INFO

```
=====
<class 'pandas.DataFrame'>
RangeIndex: 1197 entries, 0 to 1196
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   product_id      1197 non-null  int64
1   category        1197 non-null  str
2   name            1197 non-null  str
3   price_usd       1197 non-null  float64
4   cost_usd        1197 non-null  float64
5   margin_usd      1197 non-null  float64
dtypes: float64(3), int64(1), str(2)
memory usage: 56.2 KB
```

## REVIEWS - INFO

```
=====
<class 'pandas.DataFrame'>
RangeIndex: 10780 entries, 0 to 10779
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
```

```

---  -----  -----  ---
0  review_id    10780 non-null int64
1  order_id     10780 non-null int64
2  product_id   10780 non-null int64
3  rating       10780 non-null int64
4  review_text  10780 non-null str
5  review_time  10780 non-null str
dtypes: int64(4), str(2)
memory usage: 505.4 KB

```

```

In [36]: # Calcula el número y porcentaje de valores nulos por columna en todos los datasets.
null_summary = pd.DataFrame()

for name, df in datasets.items():
    temp = df.isnull().sum().reset_index()
    temp.columns = ["column", "nulls"]
    temp["dataset"] = name
    temp["null_pct"] = (temp["nulls"] / len(df) * 100).round(2)
    null_summary = pd.concat([null_summary, temp], ignore_index=True)

null_summary.sort_values(["null_pct", "nulls"], ascending=False).head(30)

```

Out[36]:

|    | column           | nulls  | dataset   | null_pct |
|----|------------------|--------|-----------|----------|
| 20 | payment          | 727378 | events    | 95.59    |
| 21 | discount_pct     | 727378 | events    | 95.59    |
| 22 | amount_usd       | 727378 | events    | 95.59    |
| 19 | cart_size        | 716049 | events    | 94.10    |
| 18 | qty              | 617832 | events    | 81.19    |
| 17 | product_id       | 78489  | events    | 10.31    |
| 0  | customer_id      | 0      | customers | 0.00     |
| 1  | name             | 0      | customers | 0.00     |
| 2  | email            | 0      | customers | 0.00     |
| 3  | country          | 0      | customers | 0.00     |
| 4  | age              | 0      | customers | 0.00     |
| 5  | signup_date      | 0      | customers | 0.00     |
| 6  | marketing_opt_in | 0      | customers | 0.00     |
| 7  | session_id       | 0      | sessions  | 0.00     |
| 8  | customer_id      | 0      | sessions  | 0.00     |
| 9  | start_time       | 0      | sessions  | 0.00     |
| 10 | device           | 0      | sessions  | 0.00     |
| 11 | source           | 0      | sessions  | 0.00     |
| 12 | country          | 0      | sessions  | 0.00     |
| 13 | event_id         | 0      | events    | 0.00     |
| 14 | session_id       | 0      | events    | 0.00     |
| 15 | timestamp        | 0      | events    | 0.00     |

|    | column         | nulls | dataset | null_pct |
|----|----------------|-------|---------|----------|
| 16 | event_type     | 0     | events  | 0.00     |
| 23 | order_id       | 0     | orders  | 0.00     |
| 24 | customer_id    | 0     | orders  | 0.00     |
| 25 | order_time     | 0     | orders  | 0.00     |
| 26 | payment_method | 0     | orders  | 0.00     |
| 27 | discount_pct   | 0     | orders  | 0.00     |
| 28 | subtotal_usd   | 0     | orders  | 0.00     |
| 29 | total_usd      | 0     | orders  | 0.00     |

```
In [37]: # Guarda Los resúmenes generados en archivos CSV dentro de reports/tables.
summary.to_csv(TABLES_DIR / "dataset_summary.csv", index=False)
null_summary.to_csv(TABLES_DIR / "null_summary.csv", index=False)

print("Archivos guardados en:")
print(TABLES_DIR / "dataset_summary.csv")
print(TABLES_DIR / "null_summary.csv")
```

Archivos guardados en:

G:\ProyectosPython\proyecto3\desarrollo3\reports\tables\dataset\_summary.csv

G:\ProyectosPython\proyecto3\desarrollo3\reports\tables\null\_summary.csv

## Resultado de esta etapa

En este notebook se dejó configurado el entorno del proyecto, se definieron rutas reproducibles, se verificó la existencia de los archivos fuente y se realizó una carga inicial de todos los datasets.

## Siguiente paso

El siguiente notebook ( `02-jj-eda-clientes.ipynb` ) se enfocará en el análisis exploratorio, calidad de datos y relaciones iniciales entre tablas.